Guía de Gestión de Proyectos: Jira, Ágil e Integración Continua

Esta guía establece los fundamentos esenciales para la gestión de proyectos de software en la industria actual. Aborda las diferencias metodológicas, la especificación de requerimientos de forma orientada a valor y la utilización de **Jira** en conjunción con **GitHub** para establecer un flujo de vida del código transparente, centralizado y profesional.

El objetivo de este material es transicionar de la programación puramente empírica hacia una ingeniería de software estructurada, donde cada línea de código tenga un propósito de negocio claramente trazado y auditado.

Índice

1. Metodologías de Desarrollo: Tradicional vs. Ágil
2. Especificación: Casos de Uso vs. Historias de Usuario
3. Jira e Implementación de Tareas
4. Integración Continua: Jira + GitHub

1. Metodologías de Desarrollo: Tradicional vs. Ágil

Al enfrentar la construcción de un producto de software, los equipos deben regirse bajo un marco de trabajo que orqueste sus actividades diarias.

1.1 Desarrollo Tradicional (Enfoque en Cascada o "Waterfall")

El modelo clásico. Concibe la construcción de software bajo fases estrictamente secuenciales (Requisitos, Diseño, Implementación, Verificación, Mantenimiento). No se avanza a la siguiente etapa sin haber documentado y cerrado por completo la anterior.

- **Pros:**
  - Genera una documentación sumamente exhaustiva.
  - Los costos iniciales, roles y tiempos de entrega teóricos son fijos y predecibles.
  - Funciona bien en sistemas de misión crítica o gubernamentales con alcances inmutables.

- **Contras:**
  - Extrema rigidez. Cualquier cambio de idea del cliente rompe toda la estructura.
  - El cliente no ve un producto funcional hasta las etapas finales del desarrollo, lo que casi siempre desencadena grandes decepciones respecto al producto que él realmente imaginaba.

1.2 Desarrollo Ágil (Scrum / Kanban)

Nació como respuesta a la rigidez del modelo tradicional. Se basa en ciclos cortos, iterativos e incrementales (Sprints). En lugar de entregar un megaproducto inmenso al final de un año, se entregan pequeñas piezas completamente funcionales y útiles cada par de semanas.

- **Pros:**
  - Excelente adaptabilidad al cambio. Permite corregir el rumbo rápidamente.
  - El cliente y el equipo dialogan constantemente inspeccionando incrementos reales. El valor se entrega de inmediato.
- **Contras:**
  - Si el equipo carece de disciplina, la documentación tiende a volverse inexistente.
  - Requiere un altísimo grado de comunicación y compromiso; no es apto para entornos de trabajo aislados.

> **Tip**
>
> Actualmente, la abrumadora mayoría de la industria y las empresas modernas (*Startups*, corporativos tecnológicos) utilizan metodologías Ágiles para sobrevivir a los rápidos cambios del mercado.

2. Requerimientos: Casos de Uso vs. Historias de Usuario

Todo desarrollo parte de una necesidad. La manera de redactar y estructurar dicha necesidad determinará cómo el equipo asume el reto técnico.

2.1 Casos de Uso (Enfoque de Sistema)

Pertenecen comúnmente a los enfoques tradicionales. Describen minuciosamente todas las interacciones entre un *Actor* (usuario, u otro programa) y el *Sistema*. Definen entradas, flujos básicos, flujos alternos y caminos de error. Su redacción es más técnica y determinista, enfocándose fuertemente en el "cómo" el sistema reacciona ante una acción.

2.2 Historias de Usuario (Enfoque de Valor)

Son el pilar fundamental del entorno Ágil. Su meta no es ser una documentación técnica gigante, sino **provocar una conversación** entre los desarrolladores y la parte interesada. Se enfocan netamente en el valor entregado y se escriben en lenguaje de negocio respondiendo a tres factores vitales: **Quién, Qué y Por Qué**.

La estructura estándar e inquebrantable de una historia de usuario es:

- **Como** [rol del usuario]
- **Quiero** [el objetivo puntual o acción esperada]
- **Para** [la razón de negocio o beneficio que obtendré al hacerlo]

> **Note**
>
> Ejemplo práctico:
> *"**Como** estudiante universitario, **quiero** poder filtrar la lista de mis tareas pendientes por fecha de vencimiento, **para** priorizar mis entregables urgentes sin fallar al semestre."*

3. Jira e Implementación de Tareas

Jira (de Atlassian) es la herramienta líder global para rastrear este tipo de progresos, planificar sprints y llevar el registro de incidencias (bugs).

3.1 Trasladando el backlog

Las Historias de Usuario previamente explicadas se introducen allí en forma de *Tickets*. Usualmente un tablero Jira (tipo Kanban) tiene columnas indicando el tránsito de valor: **To Do** (Por hacer), **In Progress** (En curso), **In Review** (En revisión) y **Done** (Terminado).

Al mover un ticket entre columnas, todo el equipo sabe precisamente y en tiempo real qué partes del sistema se están desarrollando en ese instante, democratizando la información del proyecto.

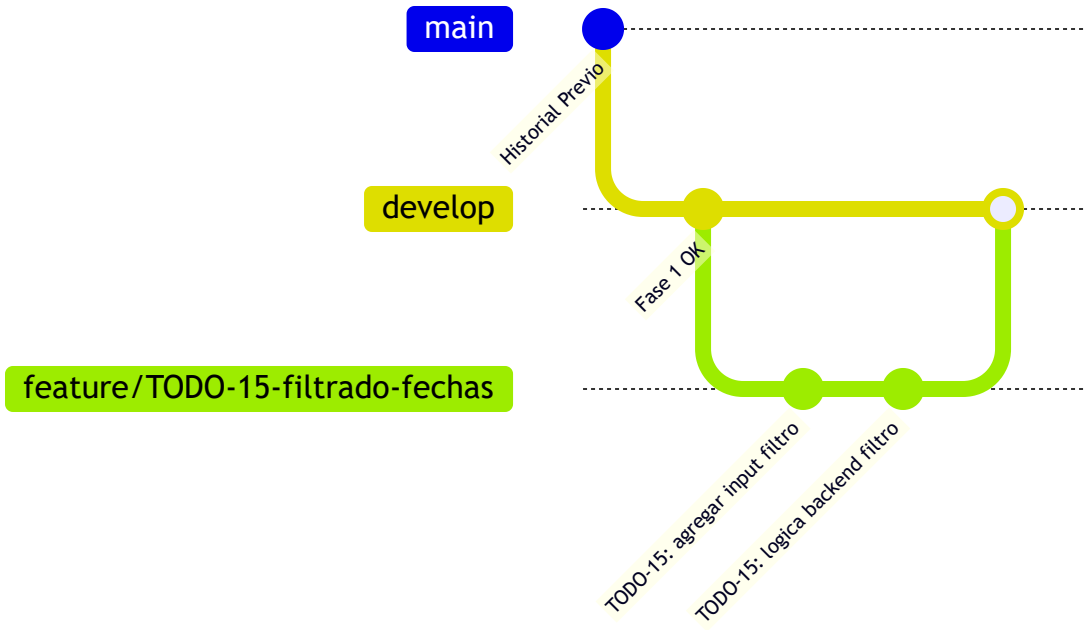
4. Integración Continua: Jira + GitHub

La visibilidad es la cualidad de los proyectos maduros. Para que compañeros, profesores o administradores auditen el trabajo real, Jira se integra a GitHub, correlacionando el código duro con las tarjetas de tablero.

El vínculo absoluto se genera a partir de la **Clave del Ticket**, el identificador principal originado por Jira. Por ejemplo: `DEV-42` o `TODO-15` .

4.1 Creación de Ramas Automatizadas

En GitFlow, nuestra rama derivará de un requerimiento estricto. Desde la propia interfaz de Jira, usted puede crear la rama que impactará en su repositorio remoto, usando esa convención formal.



4.2 Automatización y Transparencia del Flujo de Trabajo

Vincular correctamente Jira con GitHub otorga potencias industriales avanzadas:

1. **Smart Commits:** Si el desarrollador incluye la clave del ticket en todos sus commits (Ej: `git commit -m "feat(TODO-15): diseño visual para el filtro de fechas"`), Jira capturará esa

información y la insertará directamente en el detalle de la tarea en su plataforma web. Cualesquier miembro puede ver los commits ligados sin accesar directamente a GitHub.

2. **Sincronización de Pull Requests:** Al abrir un PR en GitHub apuntando hacia *develop* para fusionar la `feature/TODO-15-filtrado-fechas`, la tarjeta en el entorno visual de Jira puede transicionar de forma automática de `In Progress` a `In Review`. Cuando el profesor o líder apruebe (mergee) el código, automáticamente pasará a la columna `Done`.
3. **Visibilidad de Testing (CI/CD):** Cuando se inyectan pruebas unitarias (Testing validativo) en nuestra tubería configurada, Jira leerá el estatus general devuelto por GitHub Actions u otra matriz. En la misma tarjeta, aparecerá un checkmark o aspa roja indicando que "se probó con éxito" o que ese cambio "rompe compilaciones", impidiendo pasar tareas problemáticas a Done.

Tener este control cruzado permite auditar fehacientemente quién hizo qué, y a través de qué líneas de código se logró proveer valor a los clientes.

Guía de Integración GitHub + Jira (Scrum & GitFlow)

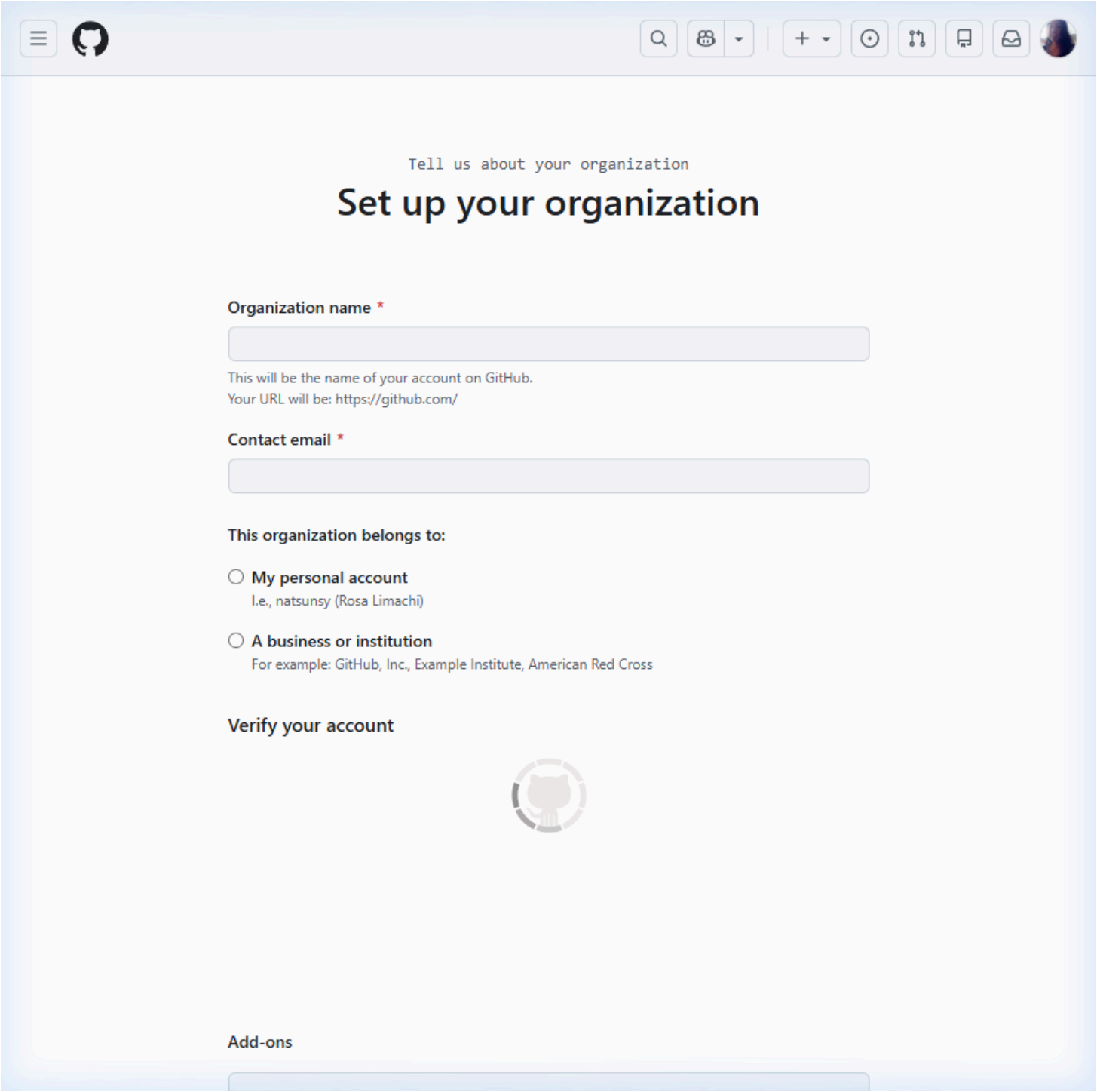
Esta guía es un tutorial detallado para configurar un entorno de desarrollo profesional. Sigue cada paso y utiliza las capturas de pantalla como referencia para asegurar que tu integración sea exitosa.

Fase 1: Preparación del Entorno en GitHub

Paso 1: Creación de la Organización

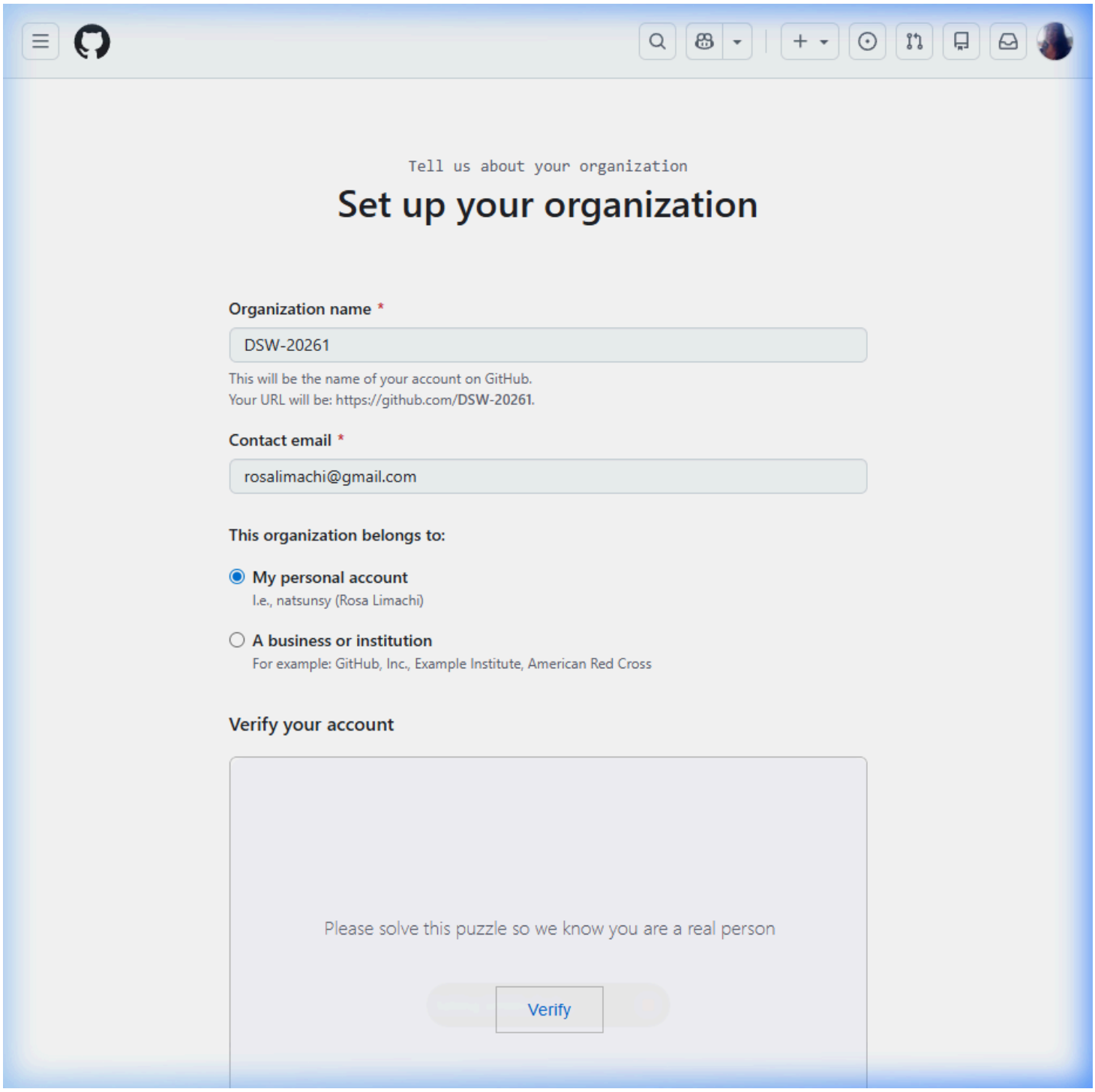
En lugar de repositorios personales, usaremos una **Organization** para el trabajo grupal.

1. Ve a tu perfil de GitHub > **Organizations** > **New organization**.
2. Elige el plan **Free**.
3. Configura el nombre (ej. `DSW-20261`) y el correo de contacto.



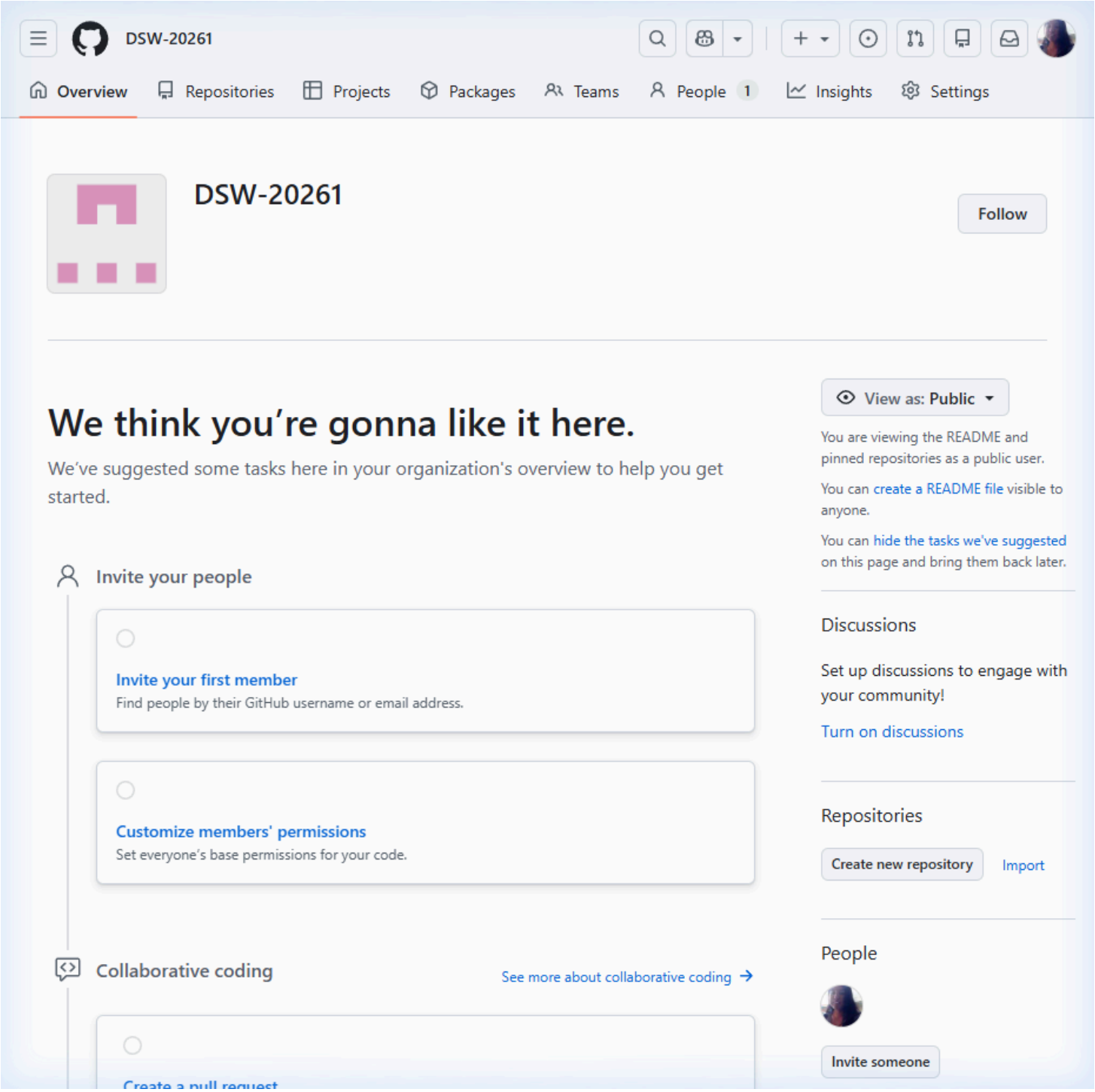
Paso 2: Verificación de Seguridad

GitHub solicitará una verificación (Captcha) para confirmar que eres humano. Complétalo para activar la organización.



Paso 3: Dashboard de la Organización

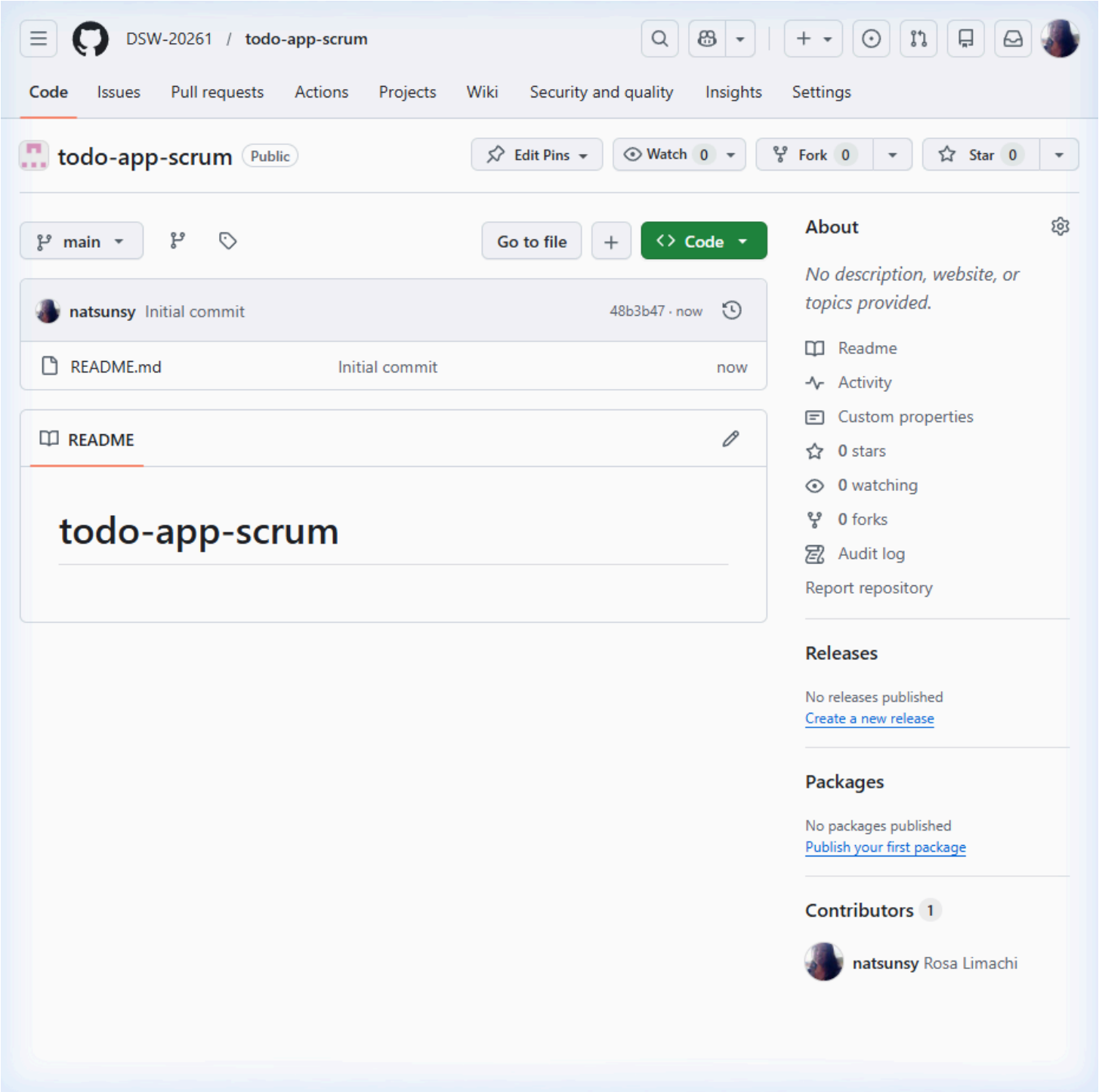
Una vez creada, llegarás al listado de tu nueva organización. Desde aquí podrás invitar a tus compañeros en la pestaña **People**.



Paso 4: Creación del Repositorio Grupal

Dentro de la organización, crea el repositorio principal. Asegúrate de:

- Ponerle un nombre descriptivo: `todo-app-scrum`.
- Marcarlo como **Public**.
- Añadir un archivo **README**.

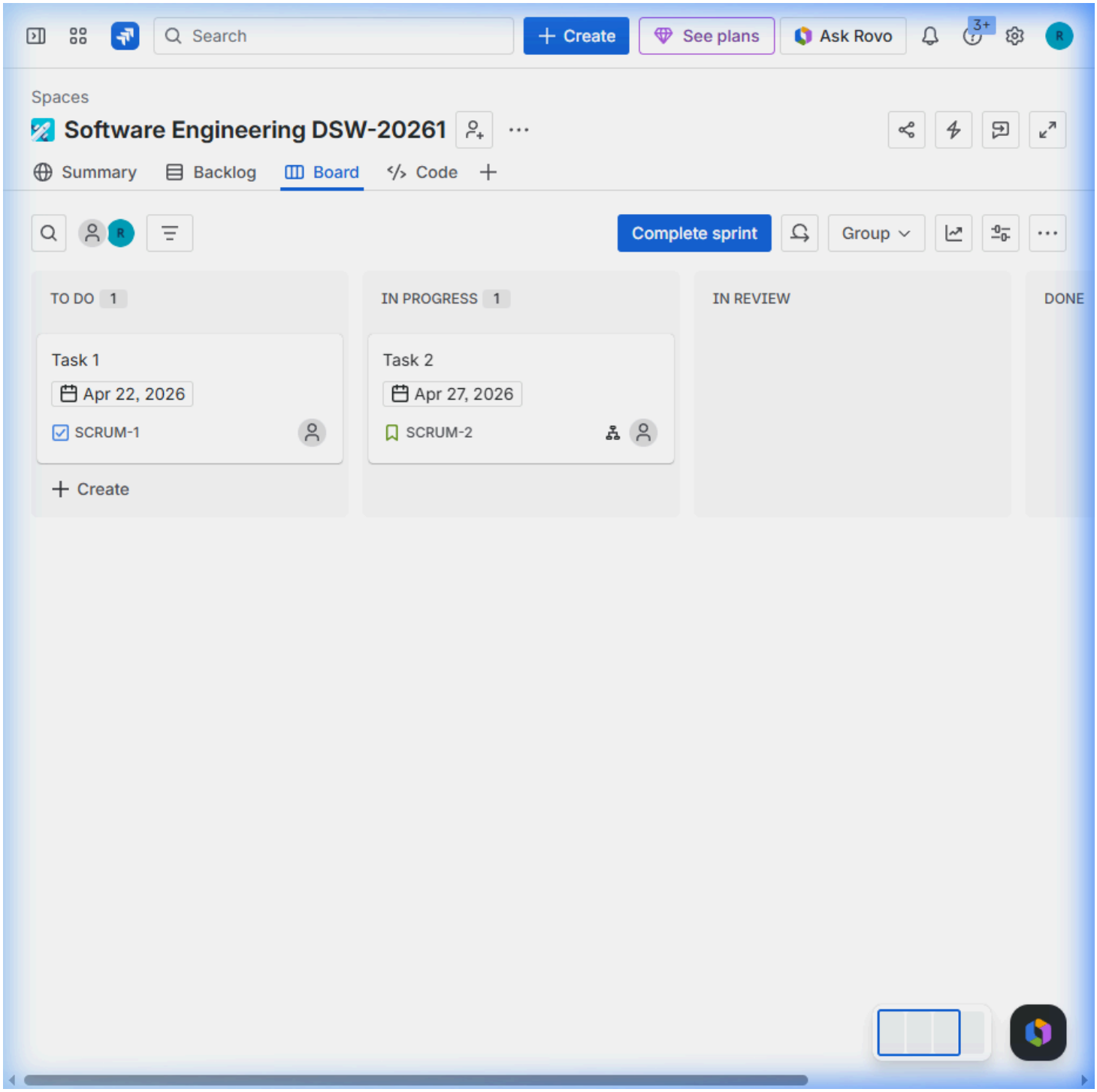


Fase 2: Configuración de Jira Software

Paso 5: Creación del Proyecto Scrum

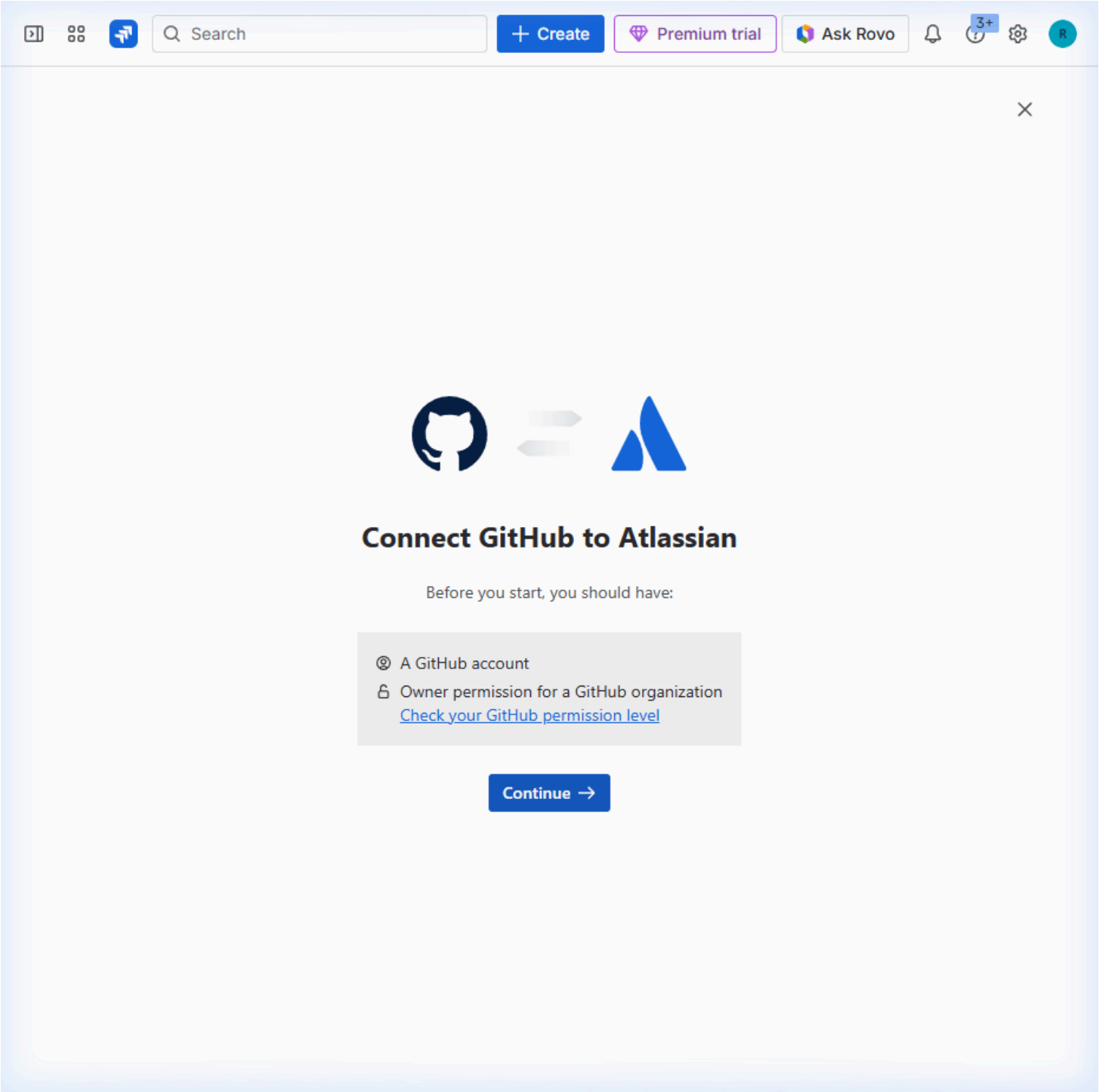
1. En Jira, crea un nuevo Space de **Software Engineering DSW-20261**.
2. Selecciona la plantilla **Scrum**.

3. (Opcional pero Recomendado): Cambia la **Key** del proyecto a `DSW` en la configuración de detalles para que tus tareas tengan identificadores como `DSW-1`.



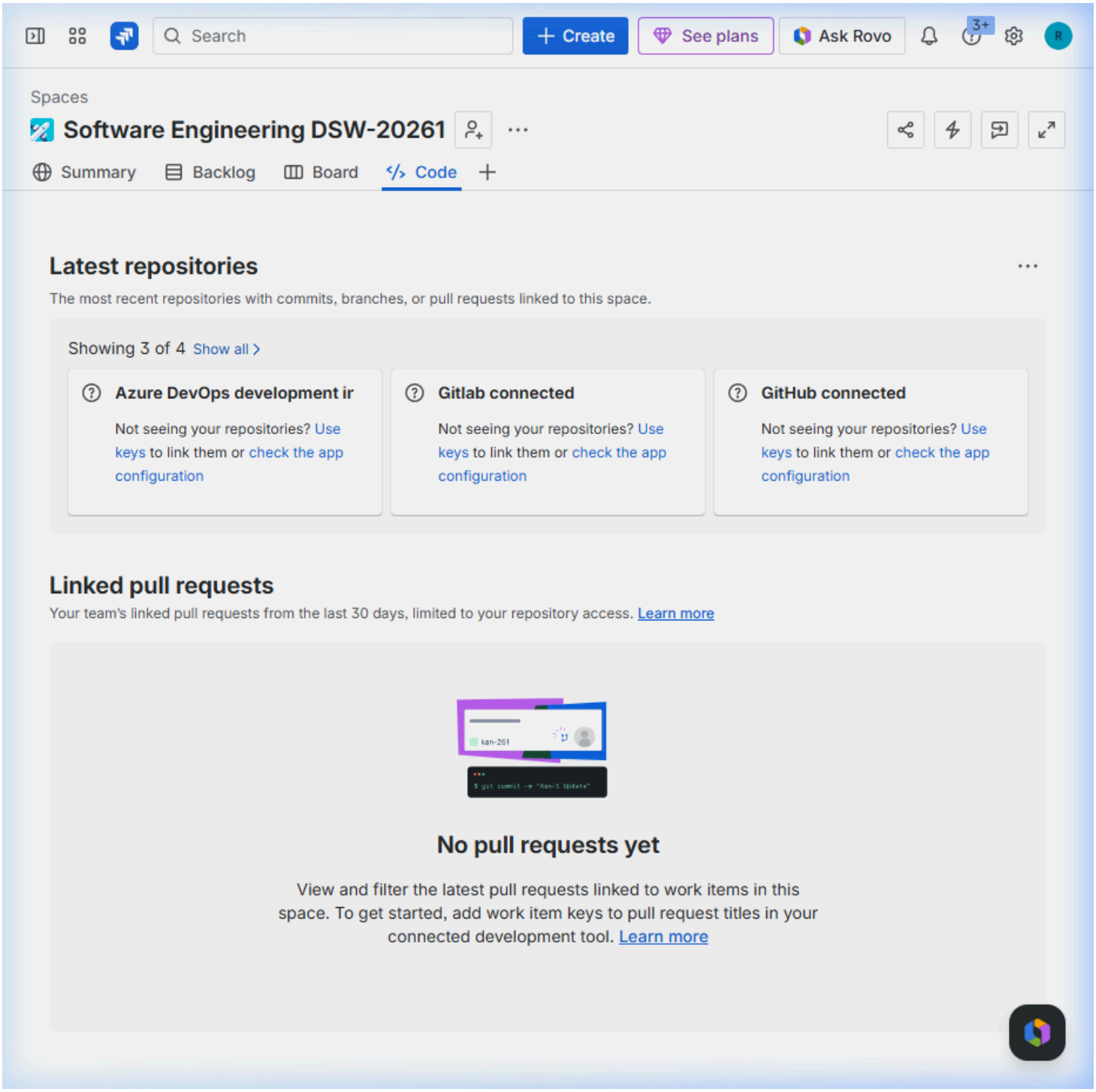
Paso 6: Instalación del App de GitHub

Navega al **Marketplace** de Jira, busca e instala **"GitHub for Jira"**. Atlassian te confirmará la instalación exitosa.



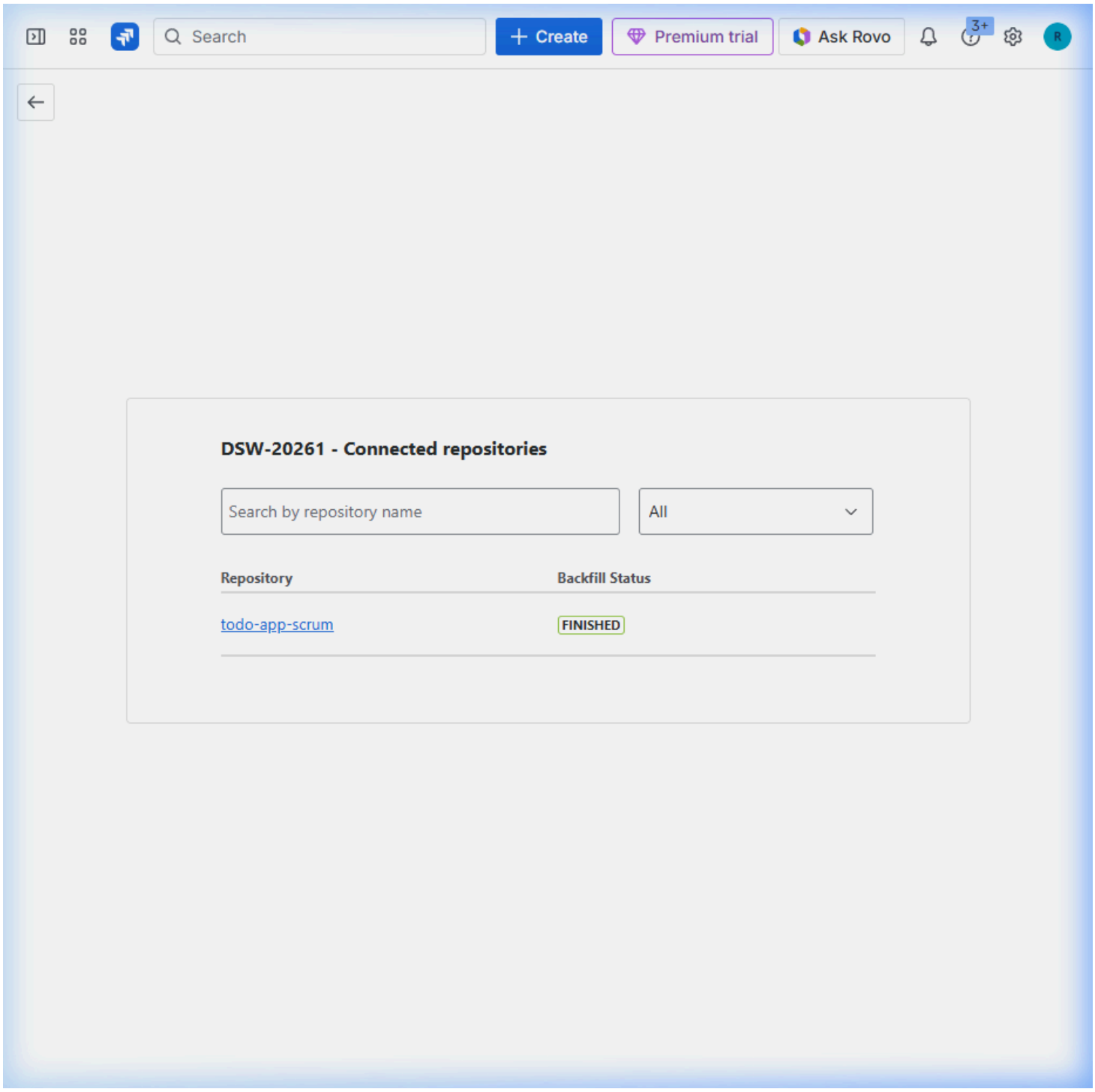
Paso 7: Configuración de la Conexión

Vincula tu organización de GitHub con Jira. Deberás autorizar el acceso y seleccionar la organización `DSW-20261` .



Paso 8: Confirmación de Sincronización

Verifica que el repositorio `todo-app-scrum` aparezca con el estado **FINISHED** o un check verde. Esto indica que Jira ya puede "leer" lo que pasa en GitHub.



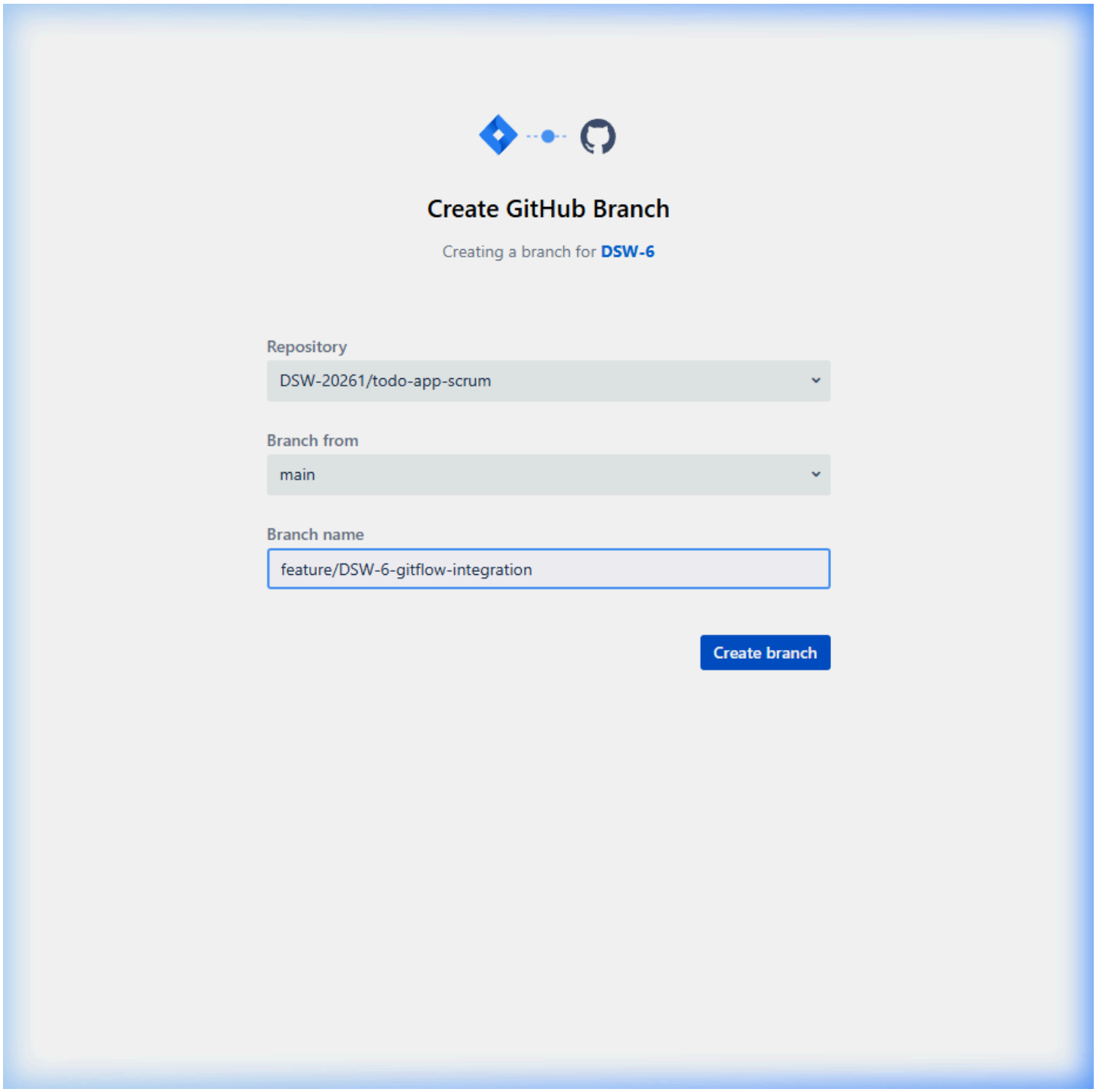
Fase 3: Flujo de Desarrollo (GitFlow)

Paso 9: Creación de Ramas desde Jira

Al trabajar en un ticket (ej. `DSW-6`), usa el botón **Create branch**. Jira sugerirá un nombre estándar.

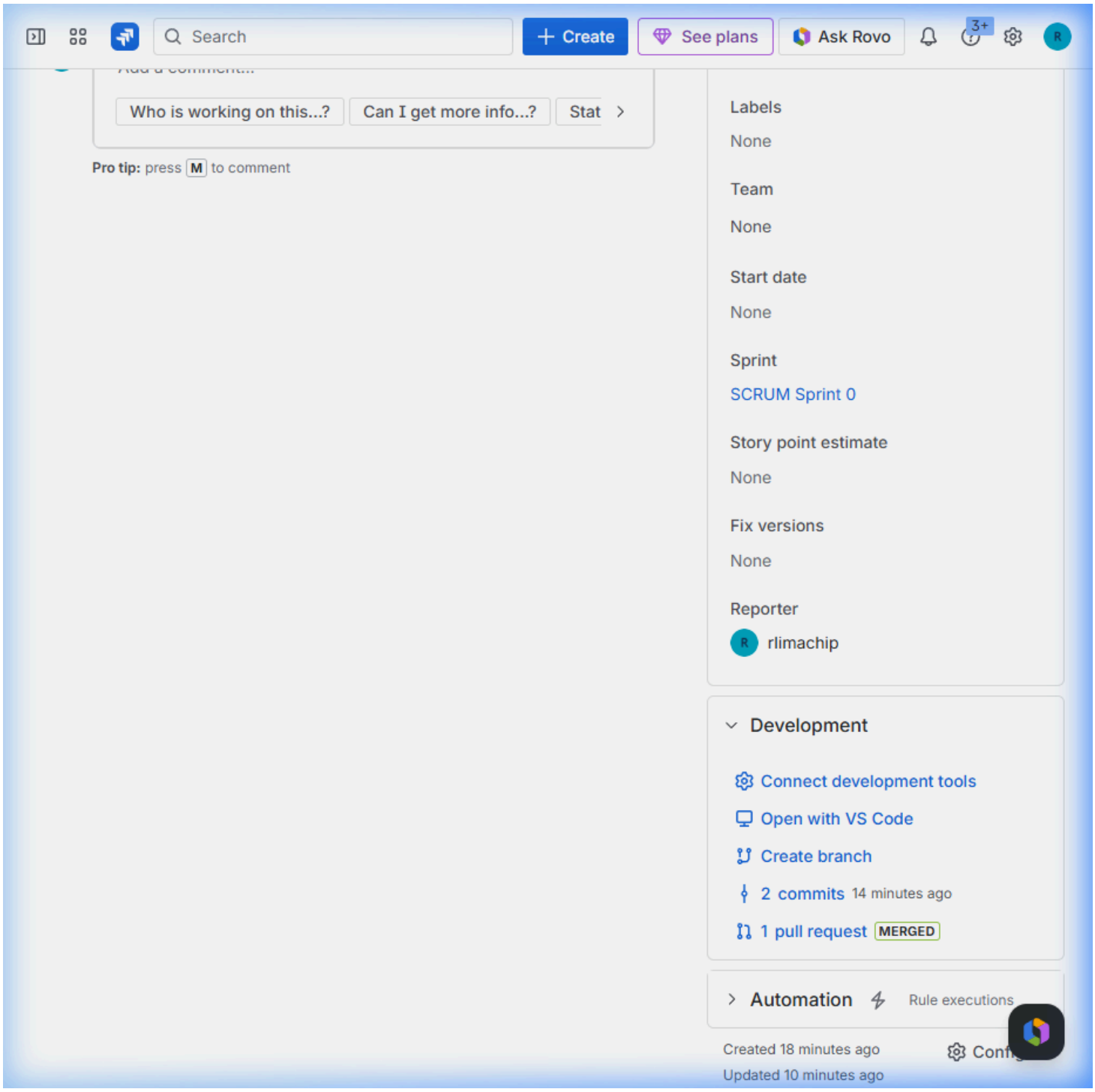
Important

- **Ajuste GitFlow**: Debes editar el nombre manualmente para añadir el prefijo `feature/`, `bugfix/` o `hotfix/`.



Paso 10: Trazabilidad en el Ticket

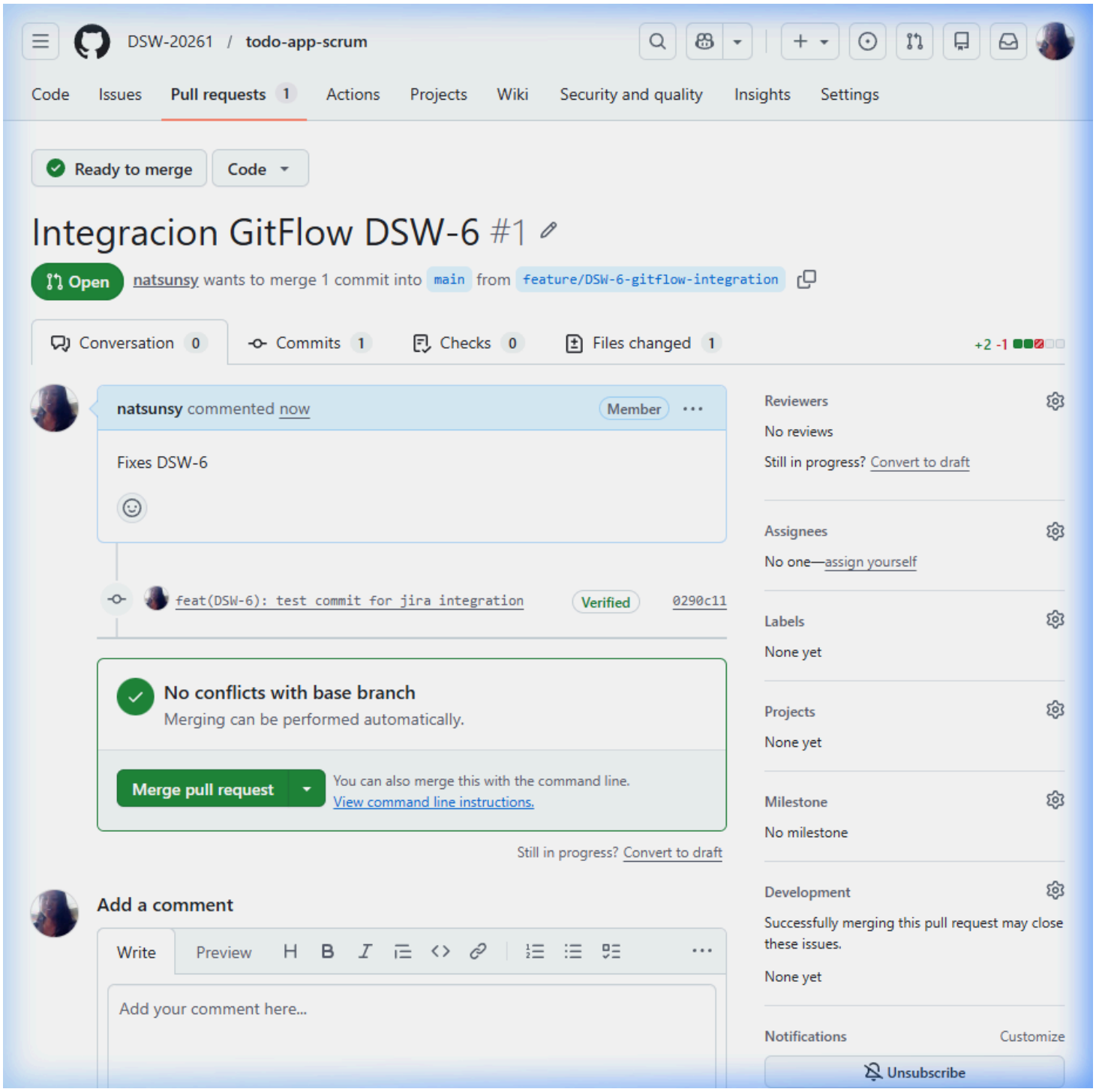
Una vez creada la rama y realizados los commits (incluyendo la key `DSW-6` en el mensaje), el panel de Jira mostrará los datos sincronizados en tiempo real.



Fase 4: Automatización y Cierre

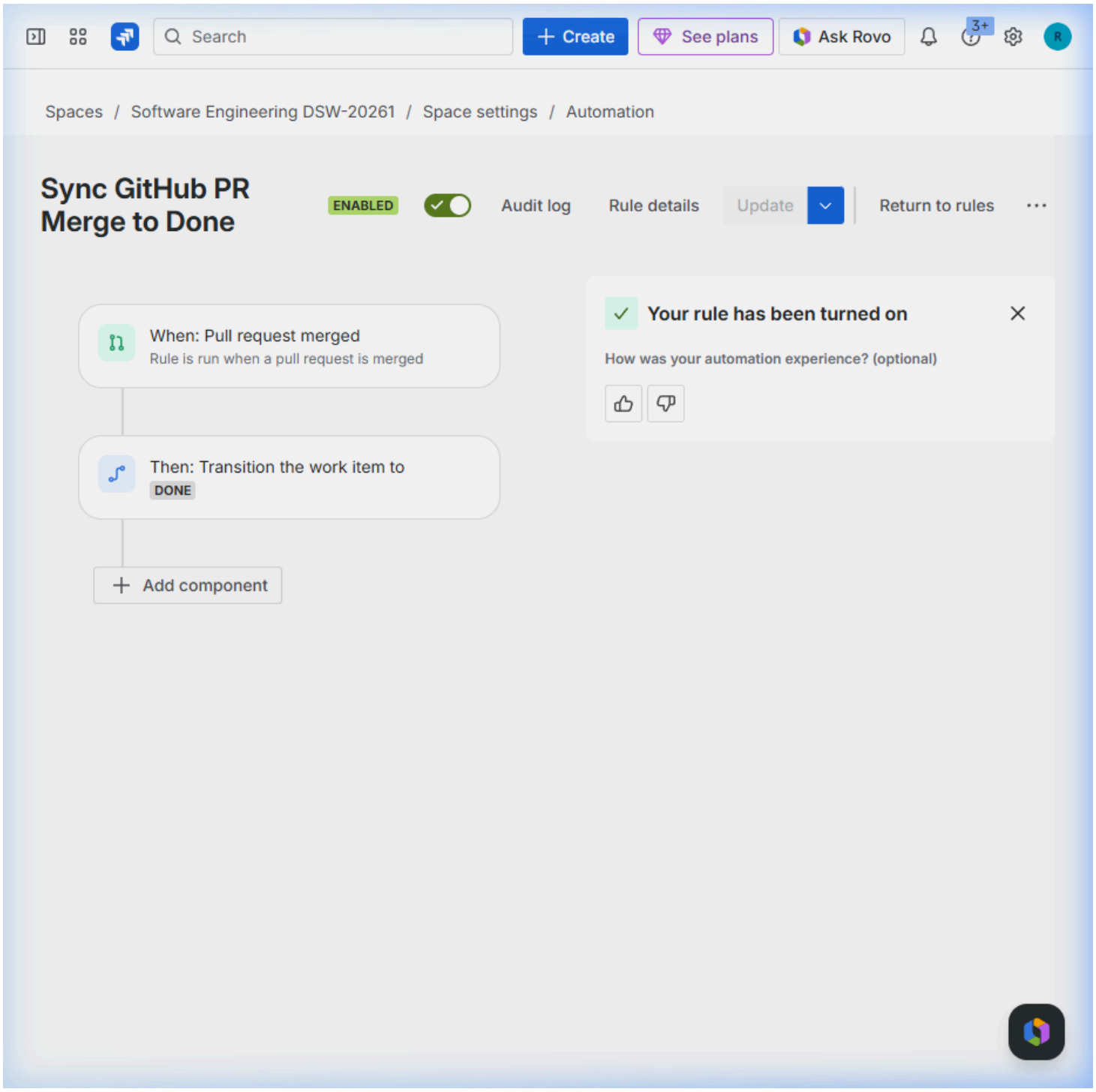
Paso 11: Creación del Pull Request (PR)

En GitHub, abre un PR para integrar tu rama `feature/*` a `main`. Menciona el ticket con `Fixes DSW-6`.



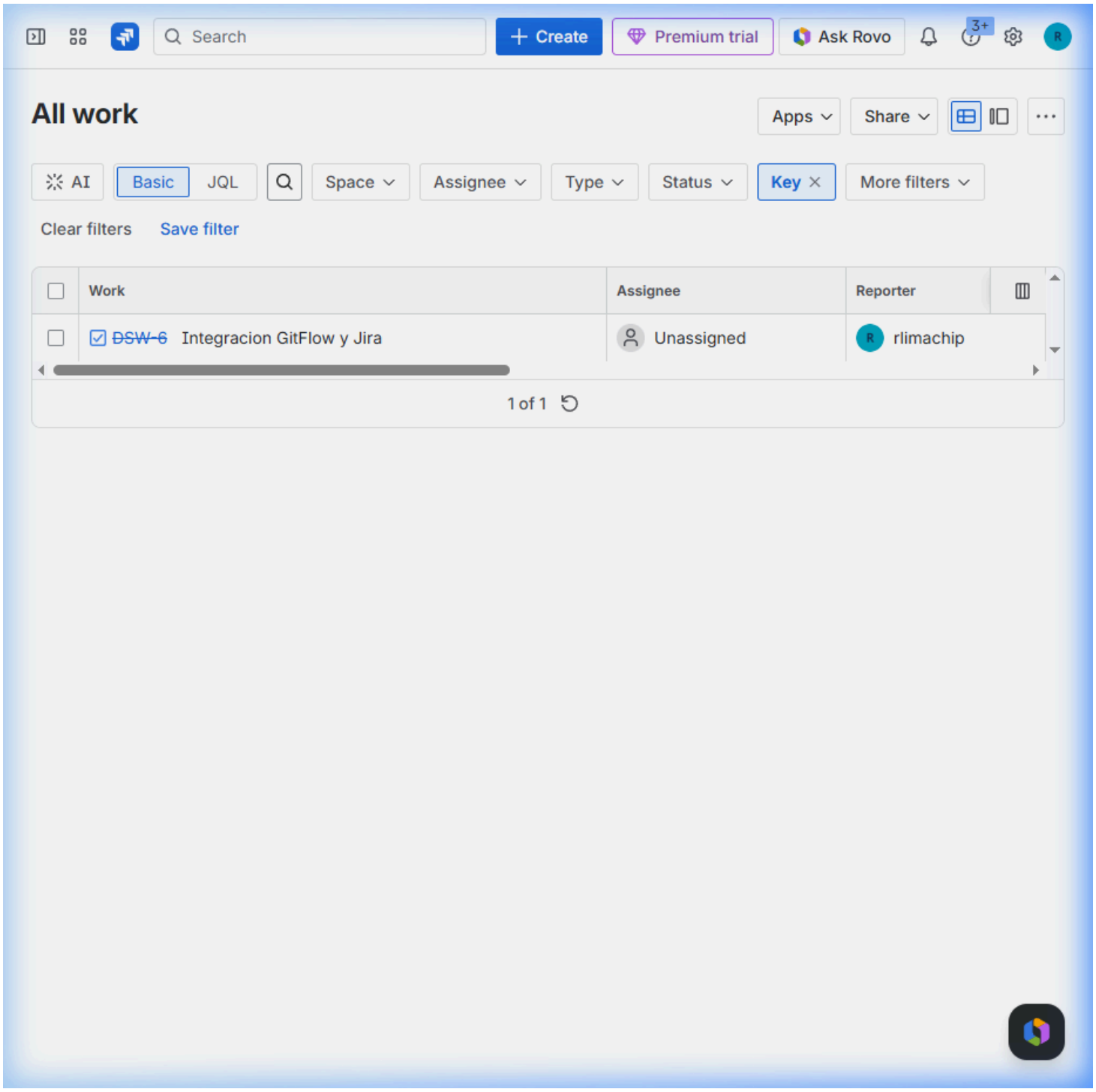
Paso 12: Configuración de la Regla de Automatización

En Jira, ve a **Automation** y crea una regla para que cuando un **Pull Request se fusione (Merged)**, el estado de la tarea pase automáticamente a **DONE**.



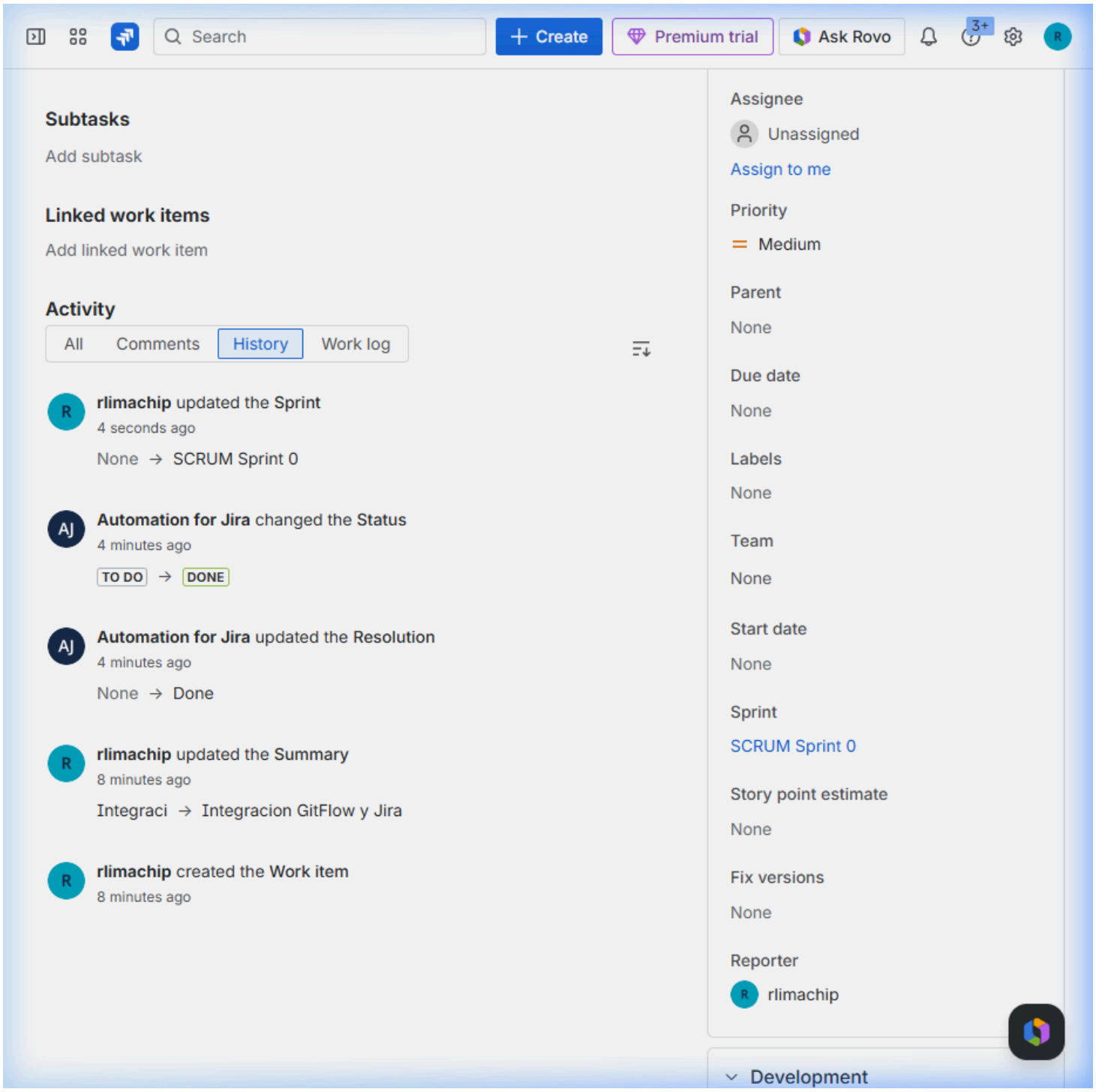
Paso 13: Tablero Actualizado

Al hacer el Merge en GitHub, notarás que en el tablero de Jira el ticket se mueve solo a la columna final.



Paso 14: Auditoría de Historial

Si revisas el **History** de la tarea, verás el log indicando que la automatización realizó el movimiento.



Conclusión

Siguiendo estos pasos, has garantizado el flujo de vida del software más utilizado en la industria: **Requerimiento en Jira -> Rama en GitHub -> Pull Request de Revisión -> Cierre Automático.**